

Orbis Scheduler

Database Schema — Entity Reference Document

Hussain Jameel · ITA602 · Torrens University Australia

Overview

The Orbis Scheduler database consists of 7 tables. All tenant data is scoped to a business UUID. Customers are guests — they have no user accounts. Business owners register and await admin approval before accessing the platform.

Tables

users — *platform login accounts for business owners and admins*

Column	Type	Notes
id	SERIAL	Primary key
name	VARCHAR(100)	Not null
email	VARCHAR(255)	Unique, not null
password_hash	VARCHAR(255)	bcrypt hashed, not null
role	ENUM	owner admin
is_active	BOOLEAN	Default true — admin can deactivate login
created_at	TIMESTAMP	Default now()

businesses — *each registered tenant on the platform*

Column	Type	Notes
id	UUID	Primary key — used in public booking URL
user_id	INTEGER	FK → users.id
name	VARCHAR(255)	Business display name, not null
slug	VARCHAR(255)	Unique, URL-friendly — e.g. joes-plumbing
description	TEXT	Nullable — shown on booking page
phone	VARCHAR(20)	Nullable
contact_email	VARCHAR(255)	Nullable — separate from login email
website_url	VARCHAR(255)	Nullable
is_active	BOOLEAN	Default true — admin suspend/activate
approval_status	ENUM	pending approved rejected
rejection_reason	TEXT	Nullable — admin fills on rejection
created_at	TIMESTAMP	Default now()

availability_rules — *weekly recurring schedule per business*

Column	Type	Notes
id	SERIAL	Primary key

Column	Type	Notes
business_id	UUID	FK → businesses.id
day_of_week	SMALLINT	0 = Monday, 6 = Sunday
start_time	TIME	Not null — e.g. 09:00
end_time	TIME	Not null — e.g. 17:00
break_start	TIME	Nullable — e.g. 12:00
break_end	TIME	Nullable — e.g. 13:00
slot_duration_minutes	SMALLINT	Not null — e.g. 30 or 60
is_available	BOOLEAN	Default true
updated_at	TIMESTAMP	Default now()

booking_forms — *custom booking form configuration per business*

Column	Type	Notes
id	SERIAL	Primary key
business_id	UUID	FK → businesses.id
title	VARCHAR(255)	Not null — e.g. Book a Consultation
description	TEXT	Nullable — shown above the form
booking_window_days	SMALLINT	Default 30 — how far ahead customers can book
is_active	BOOLEAN	Default true — only one active form per business
created_at	TIMESTAMP	Default now()
updated_at	TIMESTAMP	Default now()

form_fields — *individual fields inside a booking form*

Column	Type	Notes
id	SERIAL	Primary key
form_id	INTEGER	FK → booking_forms.id
label	VARCHAR(255)	Not null — e.g. Choose your suburb
field_type	ENUM	text textarea dropdown checkbox radio
options	JSONB	Nullable — array of options for dropdown/radio/checkbox
is_required	BOOLEAN	Default false
display_order	SMALLINT	Controls render order on the form
created_at	TIMESTAMP	Default now()

bookings — *customer booking submissions*

Column	Type	Notes
id	SERIAL	Primary key

Column	Type	Notes
business_id	UUID	FK → businesses.id — for direct tenant queries
form_id	INTEGER	FK → booking_forms.id
customer_name	VARCHAR(255)	Not null
customer_email	VARCHAR(255)	Not null
customer_phone	VARCHAR(20)	Nullable
booking_date	DATE	Not null — date customer selected
booking_time	TIME	Not null — slot customer selected
status	ENUM	pending approved rejected cancelled
owner_notes	TEXT	Nullable — owner adds notes on approval/rejection
created_at	TIMESTAMP	Default now()
updated_at	TIMESTAMP	Default now() — updates on any status change

booking_field_values — customer answers to dynamic form fields

Column	Type	Notes
id	SERIAL	Primary key
booking_id	INTEGER	FK → bookings.id
form_field_id	INTEGER	FK → form_fields.id
value	TEXT	Customer's answer — stored as text regardless of field type

 Future: file to be added to form_fields.field_type ENUM with a booking_attachments table for document and image uploads post-MVP.

 Future: booking_field_values extended with a booking_attachments table referencing file storage (MinIO/S3) when file upload is implemented.

Relationships

From		To	Description
users.id	→	businesses.user_id	One user owns one business
businesses.id	→	availability_rules.business_id	One business has 7 availability rows
businesses.id	→	booking_forms.business_id	One business has one active form
businesses.id	→	bookings.business_id	Direct tenant scoping on bookings
booking_forms.id	→	form_fields.form_id	One form has many fields
booking_forms.id	→	bookings.form_id	Bookings reference the form used
bookings.id	→	booking_field_values.booking_id	One booking has many field answers
form_fields.id	→	booking_field_values.form_field_id	Each answer references its field

Key Design Decisions

Decision	Reasoning
UUID for businesses.id	Public booking URLs use the business UUID making them unguessable. All other tables use SERIAL integers.
Slug on businesses	Provides a clean human-readable booking URL (e.g. /book/joes-plumbing) alongside the UUID.
JSONB for form_fields.options	Stores dropdown, radio and checkbox option arrays natively in PostgreSQL without requiring a separate options table.
Guests only for customers	Customers have no accounts. Their identity is stored directly on the bookings row. Reduces attack surface and system complexity.
Slots calculated on demand	Available time slots are not stored in the database. They are calculated at request time from availability_rules. No stale data, no cleanup needed.
approval_status on businesses	Businesses self-register but cannot access the platform until an admin approves them. Prevents unauthorised access without requiring manual account creation.
business_id on bookings	Denormalised FK for direct tenant-scoped queries on bookings without joining through booking_forms. Intentional redundancy for performance and clarity.

ERD Diagram:

